

Formats

When reading or writing you specify the format by calling `format(formatName)` on the `DataFrameWriter/DataFrameReader`. Format specific parameters, such as number of records to be sampled for JSON, are specified by either providing a map of options with `options` or setting option-by-option with `option` on the reader/writer.



The first party formats `json`, `jdbc`, `orc`, and `parquet` methods directly defined on the reader/writers taking the path or connection info. These methods are for convenience only and are wrappers around the more general methods we illustrate in this chapter.

JSON

Loading and writing JSON is supported directly in Spark SQL, and despite the lack of schema information in JSON, Spark SQL is able to infer a schema for us by sampling the records. Loading JSON data is more expensive than loading many data sources, since Spark needs to read some of the records to determine the schema information. If the schema between records varies widely (or the number of records is very small), you can increase the percentage of records read to determine the schema by setting `samplingRatio` to a higher value.

Example 3-27. Load JSON data, using all (100%) of records to determine the schema

```
val df2 = sqlCtx.read.format("json").option("samplingRatio", "1.0").load(path)
```



Spark's schema inference can be a compelling reason to use Spark for processing JSON data, even if the data size could be handled on a single node.

Since our input may contain some invalid JSON records you may wish to filter out, you can also take in an RDD of strings. This allows us to load the input as a standard text file, filter out our invalid records, and then load the data into JSON. This is done by using the built-in `json` function on the `DataFrameReader`, which takes RDDs or paths. Methods for converting RDDs of regular objects are covered in [“RDDs” on page 63](#).

Example 3-28. jsonRDD load

```
val rdd: RDD[String] = input.filter(_.contains("panda"))
val df = sqlCtx.read.json(rdd)
```

JDBC

The JDBC data source represent a natural Spark SQL data source, one which supports many of the same operations. Since different database vendors have slightly different JDBC implementations, you need to add the JAR for your JDBC data sources. Since SQL field types vary as well, Spark uses `JdbcDialects` with built-in dialects for DB2, Derby, MsSQL, Mysql, Oracle, and Postgres.⁵

While Spark supports many different JDBC sources, it does not ship with the JARs required to talk to all of these databases. If you are submitting your Spark job with `spark-submit` you can download the required JARs to the host you are launching and include them by specifying `--jars` or supply the maven coordinates to `--packages`. Since the Spark shell is also launched this way, the same syntax works and you can use it to include the MySQL JDBC JAR in [Example 3-29](#).

Example 3-29. Include MySQL JDBC JAR



In earlier versions of Spark `--jars` does not include the JAR in the driver's class path. If this is the case for your cluster you must also specify the same JAR to `--driver-class-path`.

JdbcDialects allow Spark to correctly map the JDBC types to the corresponding Spark SQL types. If there isn't a `JdbcDialect` for your database vendor, the default dialect will be used which will likely work for many of the types. The dialect is automatically chosen based on the JDBC URL used.



If you find yourself needing to customize the `JdbcDialect` for your database vendor, you can look for a package or `spark-packages` or extend the `JdbcDialect` class and register your own dialect.

As with the other built-in data sources, there exists a convenience wrapper for specifying the properties required to load JDBC data. The convenience wrapper `JDBC` accepts the URL, table, and a `java.util.Properties` object for connection properties (such as authentication information). The properties object is merged with the properties that are set on the reader/writer itself. While the properties object is required, an empty properties object can be provided and properties instead specified on the reader/writer.

⁵ Some types may not be correctly implemented for all databases.

Example 3-30. Create a *DataFrame* from a JDBC data source

```
sqlContext.read.jdbc("jdbc:dialect:serverName;user=user;password=pass",
    "table", new Properties)
sqlContext.read.format("jdbc")
    .option("url", "jdbc:dialect:serverName")
    .option("dbtable", "table").load()
```

The API for saving a *DataFrame* is very similar to the API used for loading. The `save()` function needs no path since the information is already specified, just as with loading.

Example 3-31. Write a *DataFrame* to a JDBC data source

```
df.write.jdbc("jdbc:dialect:serverName;user=user;password=pass",
    "table", new Properties)
df.write.format("jdbc")
    .option("url", "jdbc:dialect:serverName")
    .option("user", "user")
    .option("password", "pass")
    .option("dbtable", "table").save()
```

In addition to reading and writing JDBC data sources, Spark SQL can also run its own **JDBC server**, which is covered later on in this chapter.

Parquet

Apache Parquet files are a common format directly supported in Spark SQL, and they are incredibly space efficient and popular. Apache Parquet’s popularity comes from a number of features, including the ability to easily split across multiple files, compression, nested types, and many others discussed in **the Parquet documentation**. Since Parquet is such a popular format, there are some additional options available in Spark for the reading and writing of Parquet files. Unlike third party data sources, these options are mostly configured on the *SQLContext*, although some can be configured on either the *SQLContext* or *DataFrameReader/Writer*.

Table 3-10. Parquet Data Source Options

SQLConf	DataFrameReader/ Writer option	Default	Purpose
spark.sql.parquet.mergeSchema	mergeSchema	False	Control if schema should be merged between partitions when reading. Can be expensive, so disabled by default in 1.5.0.

SQLConf	DataFrameReader/ Writer option	Default	Purpose
spark.sql.parquet.binaryAsString	N/A	False	Treat binary data as strings. Old versions of Spark wrote strings as binary data.
spark.sql.parquet.cacheMetadata	N/A	True	Cache parquet metadata, normally safe unless underlying data is being modified by another process.
spark.sql.parquet.compression.codec	N/A	Gzip	Specify the compression codec for use with parquet data. Valid options are uncompressed, snappy, gzip, or lzo.
spark.sql.parquet.filterPushdown	N/A	True	Push down filters to parquet (when possible) ^a .
spark.sql.parquet.writeLegacyFormat	N/A	False	Write in parquet metadata in the legacy format.
spark.sql.parquet.output.committer.class	N/A	org.apache.parquet.hadoop.ParquetOutputCommitter	Output committer used by parquet. If writing to S3 you may wish to try org.apache.spark.sql.parquet.DirectParquetOutputCommitter.

^a Pushdown means evaluate at the storage, so with parquet this can often mean skipping reading unnecessary rows or files.

Example 3-32. Read Parquet file written by an old version of Spark

```
def loadParquet(path: String): DataFrame = {
  // Configure Spark to read binary data as string, note: must be configured on SQLContext
  sqlContext.setConf("spark.sql.parquet.binaryAsString", "true")
  // Load parquet data using merge schema (configured through option)
  sqlContext.read
    .option("mergeSchema", "true")
    .format("parquet")
    .load(path)
}
```

Example 3-33. Write Parquet file with default options

```
def writeParquet(df: DataFrame, path: String) = {
  df.write.format("parquet").save(path)
}
```

Hive Tables

Interacting with Hive tables adds another option beyond the other formats. As covered in “Plain Old SQL Queries and Interacting with Hive Data” on page 56, one option for bringing in data from a Hive table is writing a SQL query against it and having the result as a DataFrame. The DataFrame’s reader and writer interfaces can also be used with Hive tables as with the rest of the data sources.

Example 3-34. Load a Hive table

```
def loadHiveTable(): DataFrame = {  
    sqlContext.read.table("pandas")  
}
```



When loading a Hive table Spark SQL will convert the metadata and cache the result, if the underlying metadata has changed you can use `sqlContext.refreshTable("tablename")` to update the metadata, or the caching can be disabled by setting `spark.sql.parquet.cacheMetadata` to `false`.

Example 3-35. Write managed table

```
def saveManagedTable(df: DataFrame): Unit = {  
    df.write.saveAsTable("pandas")  
}
```



Unless specific conditions are met, the result saved to a Hive managed table will be saved in a Spark specific format that other tools may not be able to understand.

RDDs

Spark SQL DataFrames can easily be converted to RDDs of Row objects, and can also be created from RDDs of Row objects as well as JavaBeans, Scala case classes, and tuples. For RDDs of strings in JSON format, you can use the methods discussed in “JSON” on page 59. Datasets of type T can also easily be converted to RDDs of Type T, which can provide a useful bridge for DataFrames to RDDs of concrete case classes instead of Row objects. RDDs are a special case data source, since when going to/from RDDs, the data remains inside of Spark without writing out to or reading from an external system.



Converting a DataFrame to an RDD is a transformation (not an action); however, converting an RDD to a DataFrame or Dataset may involve computing (or sampling some of) the input RDD.



Creating a DataFrame from an RDD is not free in the general case. The data must be converted into Spark SQL's internal format.

When you create a DataFrame from an RDD Spark SQL needs to add schema information. If you are creating the DataFrame from an RDD of case classes or plain old Java objects (POJOs), Spark SQL is able to use reflection to automatically determine the schema. You can also manually specify the schema for our data using the structure discussed in “Basics of Schemas” on page 41. This can be especially useful if some of our fields are not nullable. You must specify the schema yourself if Spark SQL is unable to determine the schema through reflection, such as an RDD of Row objects (perhaps from calling `.rdd` on a DataFrame to use a functional transformation).

Example 3-36. Creating DataFrames from RDDs

```
def createFromCaseClassRDD(input: RDD[PandaPlace]) = {
  // Create DataFrame explicitly using sqlContext and schema inference
  val df1 = sqlContext.createDataFrame(input)

  // Create DataFrame using sqlContext implicits and schema inference
  val df2 = input.toDF()

  // Create a Row RDD from our RDD of case classes
  val rowRDD = input.map(pm => Row(pm.name,
    pm.pandas.map(pi => Row(pi.id, pi.zip, pi.happy, pi.attributes))))

  // Create DataFrame explicitly with specified schema
  val schema = StructType(List(StructField("name", StringType, true),
    StructField("pandas", ArrayType(StructType(List(
      StructField("id", LongType, true),
      StructField("zip", StringType, true),
      StructField("happy", BooleanType, true),
      StructField("attributes", ArrayType(FloatType), true)))))))

  val df3 = sqlContext.createDataFrame(rowRDD, schema)
}
```



Case classes or JavaBeans defined inside another class can sometimes cause problems. If your RDD conversion is failing, make sure the case class being used isn't defined inside another class.

Converting a DataFrame to an RDD is incredibly simple; however, you get an RDD of Row objects. Since a row can contain anything, you need to specify the type (or cast the result) as you fetch the values for each column in the row. With Datasets you can directly get back an RDD templated on the same type, which can make the conversion back to a useful RDD much simpler.



While Scala has many implicit conversions for different numeric types, these do not generally apply in Spark SQL, instead we use explicit casting.

Example 3-37.

```
def toRDD(input: DataFrame): RDD[RawPanda] = {
  val rdd: RDD[Row] = input.rdd
  rdd.map(row => RawPanda(row.getAs[Long](0), row.getAs[String](1),
    row.getAs[String](2), row.getAs[Boolean](3), row.getAs[Array[Double]](4)))
}
```



If you know that the schema of your DataFrame matches that of another, you can use the existing schema when constructing your new DataFrame. One common place where this occurs is when an input DataFrame has been converted to an RDD for functional filter and then back.

Local Collections

Much like with RDDs, you can also create DataFrames from local collections and bring them back as local collections. The same memory requirements apply; namely, the entire contents of the DataFrame will be in memory in the driver program. As such, distributing local collections is normally limited to unit tests, or joining small data sets with larger distributed datasets.

Example 3-38. Creating from a local collection

```
def createFromLocal(input: Seq[PandaPlace]) = {
  sqlContext.createDataFrame(input)
}
```



The LocalRelation's API we used here allows us to specify a schema in the same manner as when we are converting an RDD to a DataFrame.



In pre-1.6 versions of PySpark, schema inference only looked at the first record.

Collecting data back as a local collection is more common and often done post aggregations or filtering on the data. For example, with ML pipelines collecting the coefficients or in our Goldilocks example collecting the quantiles to the driver. For larger data sets, saving to an external storage system (such as a database or HDFS) is recommended.



Just as with RDDs do not collect large DataFrames back to the driver. For Python users, it is important to remember that `toPandas()` collects the data locally.

Example 3-39. Collecting the Result locally

```
def collectDF(df: DataFrame) = {  
    val result: Array[Row] = df.collect()  
    result  
}
```

Additional Formats

As with core Spark, the data formats that ship directly with Spark only begin to scratch the surface of the types of systems with which you can interact. Some vendors publish their own implementations, and many are published on [Spark Packages](#). As of this writing there are over twenty formats listed on the [Data Source's page](#) with the most popular being [Avro](#), [Redshift](#), [CSV](#)⁵, and a unified wrapper around 6+ databases called [deep-spark](#).

Spark packages can be included in our application in a few different ways. During the exploration phase (e.g. using the shell) you can include them by specifying `--packages` on the command line, as in [Example 3-40](#). The same approach can be used when submitting our application with `spark-submit`, but this only includes the pack-

⁵ There is also a proposal to include `csv` directly in Spark since it is such a popular format.

age at run time, not at compile time. For including at compile time you can add the maven coordinates to our builds, or, if building with sbt, the `sbt-spark-package` plugin simplifies package dependencies with `spDependencies`.

Example 3-40. Starting Spark shell with CSV support

```
./bin/spark-shell --packages com.databricks:spark-csv_2.10:1.3.0
```

Example 3-41. Include spark-csv as an sbt dependency

```
"com.databricks" % "spark-csv_2.10" % "1.3.0",
```

Once you've included the package with our Spark job you need to specify the format, as you did with the Spark provided ones. The name should be mentioned in the packages documentation. For `spark-csv` you would specify a format string of `com.databricks.spark.csv`.

There are a few options if the data format you are looking for isn't directly supported in either Spark or one of the libraries. Since many formats are available as Hadoop input formats, you can try to load our data as a Hadoop input format and convert the resulting RDD as discussed in “[RDDs](#)” on [page 63](#). This approach is relatively simple, but means Spark SQL is unable to push down operations to our data store ⁵.

For a deeper integration you can implement our data source using the [Data Source API](#). Depending on which operations you wish to support operator push-down for, your base relation you will need to implement additional traits from the `org.apache.spark.sql.sources` package. The details of implementing a new Spark SQL data source are beyond the scope of this book, but if you are interested the [scala-doc for org.apache.spark.sql.sources](#) and `spark-csv`'s `CsvRelation` can be a good ways to get started.

Save Modes

In core Spark, saving RDDs always requires that the target directory does not exist, which can make appending to existing tables challenging. With Spark SQL, you can specify the desired behavior when writing out to a path that may already have data. The default behaviour is `SaveMode.ErrorIfExists`; matching RDDs behaviour, Spark will throw an exception if the target already exists. The different save modes and their behaviours are listed in [Table 3-11](#).

⁵ For example, only reading the required partitions when a filter matches one of our partitioning schemes

Table 3-11. Save modes

Save Mode	Behaviour
ErrorIfExists	Throws an exception if the target already exists. If target doesn't exist write the data out.
Append	If target already exists, append the data to it. If the data doesn't exist write the data out.
Overwrite	If the target already exists, delete the target. Write the data out.
Ignore	If the target already exists, silently skip writing out. Otherwise write out the data.

Example 3-42. Specify save mode of append

```
def writeAppend(input: DataFrame): Unit = {  
    input.write.mode(SaveMode.Append).save("output/")  
}
```

Partitions (Discovery and Writing)

Partition data is an important part of Spark SQL since it powers one of the key optimizations to allow reading only the required data, discussed more in “[Logical and Physical Plans](#)” on page 75. If you know how our downstream consumers may access our data (e.g. reading data based on zip code), when you write your data it is beneficial to use that information to partition our output. When reading the data, it's useful to understand how partition discovery functions, so you can have a better understanding of whether our filter can be pushed down.



Filter push down can make a huge difference when working with large data sets by allowing Spark to only access the subset of data required for your computation instead of doing effectively a full table scan.

When reading partitioned data, you point Spark to the root path of your data, and it will automatically discover the different partitions. Not all data types can be used as partition keys; currently only strings and numeric data are the supported types.

If our data is all in a single `DataFrame`, the `DataFrameWriter` API makes it easy to specify the partition information while you are writing the data out. The `partitionBy` function takes a list of columns to partition the output on. You can also manually save our separate `DataFrames` (say if you are writing from different jobs) with individual save calls.

Example 3-43. Save partitioned by zip code

```
def writeOutByZip(input: DataFrame): Unit = {  
    input.write.partitionBy("zipcode").format("json").save("output/")  
}
```

In addition to splitting the data by a partition key, it can be useful to make sure the resulting file sizes are reasonable, especially if the results will be used downstream by another Spark job. See ??? for general guidance.

Datasets

Datasets are an exciting extension of the DataFrame API which provide additional compile time type checking. Datasets can be used when our data can be encoded for Spark SQL and you know the type information at compile time. The Dataset API is a strongly typed collection with a mixture of relational (DataFrame) and functional (RDD) transformations. Like DataFrames, Datasets are represented by a logical plan the **Catalyst optimizer** can work with and when cached the data is stored in Spark SQL's internal encoding format.



The Dataset API is new in Spark 1.6 and will change in future versions. Users of the Dataset API are advised to treat it as a “preview.” Up-to-date documentation on the Dataset API can be found in the [scaladoc](#).

Interoperability with RDDs, DataFrames, and Local Collections

Datasets can be easily converted to/from DataFrames and RDDs, but in the initial version they do not directly extend either. Converting to/from RDDs involves encoding/decoding the data into a different form. Converting to/from DataFrames is almost “free” in that the underlying data does not need to be changed, only extra compile time type information is added/removed.



It is intended that future versions of DataFrames may extend `DataSet[Row]` to allow for easier interoperability.

To convert a DataFrame to a Dataset you can use the `as[ElementType]` function on the DataFrame to get a `Dataset[ElementType]` back as shown in [Example 3-44](#). The `ElementType` must be a case class, or similar such as tuple, consisting of types Spark SQL can represent (see “[Basics of Schemas](#)” on page 41). To create Datasets from local collections, `createDataSet(...)` on the `SQLContext` and the `toDS()` implicit